



* CNN :- Able to detect patterns in images

I/P image $\rightarrow$ Filter (Kernel) (In CNN layer) $\rightarrow$ Feature Map $\rightarrow$ Actⁿ Relu

0	1			
0	0			
0	0			

0	0	1
0	1	0
1	0	0

$\rightarrow$ Intensity of each pixel in filter is determined by backpropagation

Before training CNN, we start with random pixel values, after training with BP, we end up with smthg more useful.

Then we take out dot product betⁿ I/P and Filter - Feature detectors.

We say that Filter is convolved with I/P

$$\text{I/P} \times \text{Filter} = (0 \times 0) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 0)$$

$= 3$

Now we add bias to the o/p of the filter and put final value into Feature Map.

Then we slide over one pixel & then again dot product of I/P image & filter + bias $\rightarrow$ Feature Map.

Then we run activatⁿ functⁿ Relu that turns all neg values to 0 & pos values are same as before.

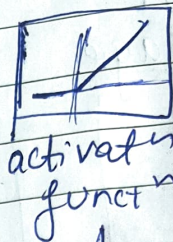
Date ___/___/___

So result will be

ReLU helps with making model non linear.

Feature Map $\rightarrow$

1	-1	-2	-1
-1	-2	-1	-2
-2	-1	-2	-1
-1	-2	-1	1

Max value in each region $\leftarrow$ Max Pooling

1	0
0	1



1	0	0	0
0	0	0	0
0	0	0	0
0	0	0	1

Feature Map, Post ReLU

or
Avg PoolingNow flattening $\rightarrow$ 

The neural network has 4 I/P nodes

Now we multiply input nodes with associated wghts & add each term together & then add bias.

So we started it as 6 by 6 - 36 potential inputs, but compressed down those to just 4 inputs into neural networks

With each convolutional layer we need to specify no. of filters the layers should have.



→ Stride - no. of pixels the filter moves at each step.

* Stride ≥ 1 → filter moves one pixel at a time.

↳ ≥ 2 → filter jumps 2 pixels (faster, less detail)

Many overlapping steps

Larger stride = ~~filter moves~~ smaller O/P size

Smaller stride = filter jumps 2 pixels.

* Padding - Adding extra pixels around the border of input.

↳ Prevents loss of edge info.

↳ Controls output size.

Output size formula. =
$$\frac{N - F + 2P}{S} + 1$$

Benefits of pooling :- Reduces dimensions & computation

- Reduce overfitting

- Model is tolerant towards variations.

→ Data Augmentation - is technique used to inc. the size & variety of training data by creating modified versions of existing images.

Eg - Zooming, flipping, rotation, shifting etc.

* Transfer Learning - technique where pre-trained model is reused for new but related task instead of training the model from scratch.

Popular Pre-trained CNN models.

AlexNet, VGG16, ResNet, MobileNet.

Step 1 Load pre-trained model

Step 2 Freeze earlier layers

Step 3 Train only final layers on new dataset

→ A filter is a small matrix with learnable weights. As it slides over input with defined stride, it produces scalar at each pos. This captures local spatial patterns like edges, corner or texture.

Key params :- Kernel Size eg. 3×3 , 5×5

Commonly used

Larger kernels see more context but cost more compute.

(i) Stride - how many pixels the filter shifts each step.

(ii) Padding - adding zeros around the border.

(iii) Depth - Multiple filters run in parallel, each learning different feature.



→ Pooling - reduces spatial dimensions of feature maps.

Benefits -

- Computationally efficient
- Controls overfitting
- Introduces degree of translatⁿ invariance

same feature detected slightly shifted still produces similar output.

↳ Max, Average

Each filter in CN layer produces a feature map
2D spatial representation

With 64 filters you get 64 feature maps stacked in depth often written as $H \times W \times 64$

Early layer vs Deep layers

Layer 1 - Very low level: horizontal edges, vertical edges, color gradients, corners

Layer 2-3 - Mid level: textures, curves, shapes, simple patterns

Deep layers - High level - semantic: eyes, wheels, faces

Date ___/___/___



* Transformer Architecture :-

Uses self attention to understand relⁿships bet. words.

High level structure - 2 parts - Encoder - Decoder

Flow :- (i) Input Embedding

Words are converted to vectors.

(ii) Positional Encoding - Transformer processes all words simultaneously, so doesn't naturally know the order of word.
L P E. adds posⁿ to the info.

(iii) Multi Head Self Attention - Each word attends to all other words.

Q, K, V - Actual info.

What am I searching for? What info do I contain

Transformer uses many attention heads. Each learns diff. relⁿships - grammar, semantics etc. subj-obj. relⁿ.

(iv) Add & normalize - Layer Norm. added

(v) Feed Forward NN - Each token independently

Date ___ / ___ / ___



Complex pattern.

(vi) Repeat Encoder Decoder part.

(vii) Final Linear + Softmax - Produces probability distribution over vocabulary

I/P $\rightarrow$ Embedding $\rightarrow$ Posⁿal Encoding $\rightarrow$ Multi-Head Attention $\rightarrow$ Feed Forward $\rightarrow$ Encoder O/P $\rightarrow$ Decoder Attention $\rightarrow$ Softmax $\rightarrow$ Generated O/P

GPT - Decoder, BERT - Encoder

Translator - I/P $\rightarrow$ Eng.

Encoder - Understands full meaning

Decoder - Uses that understanding to generate another language.